

Pipeline de Projetos com IA

Um processo com portões objetivos para construir software com agentes de inteligência artificial

Guia completo para apresentação

O que é, como funciona o fluxo, o que ele tem de diferente, como economiza contexto, o que são os agentes e qual o papel do Obsidian.

Versão do kit: **v13.4** · 24 agentes · portão automático com 14 falhas e 16 avisos · 57 testes

Documento gerado em 13 de agosto de 2026

O que você vai encontrar aqui

Este documento foi escrito para você dominar o assunto e conseguir explicá-lo a qualquer pessoa da banca, inclusive a quem nunca ouviu falar em agentes de IA. Cada termo técnico é explicado na primeira vez que aparece. Os números citados vêm de arquivos e comandos reais do projeto — quando algo é estimativa, está dito que é.

Parte	O que ela responde
1. O problema	Por que construir software com IA dá errado sem processo
2. A ideia em uma frase	O que o projeto é, em linguagem de banca
3. Vocabulário mínimo	Os seis termos que o resto do documento usa
4. A estrutura	As pastas e o que cada uma guarda
5. As 7 regras	O núcleo do método, com o motivo de cada regra
6. O fluxo	As sete fases, do dia 1 à entrega, com o portão de cada uma
7. O ciclo da sessão	O que se repete 90% do tempo
8. Os 24 agentes	O que é uma skill, quais existem e como se combinam
9. Economia de contexto	O diferencial central, com os números medidos
10. Os guardrails	O que a máquina julga sozinha e o que ela não julga
11. Rastreabilidade	D-NN, Q-NN, QA-NN e a ideia da lista-morta
12. O Obsidian	O que ele acrescenta e por que nada depende dele
13. O papel do dono	O que a IA não faz por você
14. A evidência de campo	A medição num projeto real, com o custo junto
15. Onde o kit não serve	Os limites declarados
16. Perguntas prováveis	O que a banca costuma perguntar, e a resposta
Anexo A	Glossário rápido
Anexo B	A ficha de avaliação preenchida, com justificativa

1. O problema que o projeto resolve

Quando uma pessoa usa um assistente de inteligência artificial para escrever software, o assistente não tem memória entre uma conversa e outra. Tudo o que ele precisa saber sobre o projeto tem de ser **recolocado na conversa** toda vez. Esse texto recolocado chama-se **contexto**, e ele é cobrado: quanto maior, mais caro e mais lento fica cada pedido — e, acima de certo tamanho, o modelo começa a perder detalhes no meio.

Na prática, três coisas dão errado sempre:

1. **O contexto incha sem ninguém perceber.** Cada sessão acrescenta um parágrafo "só para o agente entender melhor", e em duas semanas o arquivo que abre toda conversa virou uma parede de texto que ninguém lê e todo mundo paga.
2. **O agente regenera em vez de alterar.** Peça uma correção pequena e ele devolve o arquivo inteiro reescrito. Ninguém consegue revisar 500 linhas para achar as 3 que mudaram, então ninguém revisa — e o erro entra.
3. **As decisões se perdem.** O que foi discutido e descartado na segunda-feira volta a ser proposto na sexta, porque nada registrou que aquilo já tinha morrido.

A frase que resume

O projeto não é uma ferramenta de IA. É um **processo** que impõe disciplina a qualquer ferramenta de IA — com regras que uma máquina consegue verificar, e não apenas boas intenções escritas num documento que ninguém abre.

2. A ideia, em uma frase

Um kit reutilizável de processo que permite tirar uma aplicação do zero usando agentes de IA e sustentá-la até a entrega, mantendo rigor — portões objetivos, decisões rastreáveis, revisão adversarial — e gastando pouco contexto: orçamento numérico, leitura sob demanda, evolução por delta.

Desempacotando essa frase, porque cada pedaço dela vira uma pergunta de banca:

- **"Kit reutilizável"** — não é o código de um app. É a pasta de processo que se instala em qualquer projeto novo com um comando, levando junto as regras, os agentes e o portão automático.
- **"Portões objetivos"** — nada é aceito por "parece bom". Cada etapa tem um critério que dá para verificar: um comando que fica verde, um número que passa de um limite, um arquivo que existe.
- **"Decisões rastreáveis"** — toda decisão vira uma linha numerada (D-01, D-02...) que nunca é apagada, e que os commits do git citam.
- **"Revisão adversarial"** — antes de entregar, uma sessão separada tenta *quebrar* o que foi construído, em vez de conferir se está bonito.
- **"Gastando pouco contexto"** — é o diferencial central, e tem capítulo próprio (parte 9).

O kit está publicado no GitHub sob licença MIT e é operado dentro do Obsidian, um aplicativo gratuito de notas. Nada nele depende de uma ferramenta de IA específica: os agentes são

arquivos de texto que funcionam instalados ou simplesmente colados numa conversa.

3. Vocabulário mínimo (seis termos)

Estes seis termos aparecem o tempo todo. Se você dominar esta página, o resto do documento se lê sozinho.

Termo	O que significa, em linguagem simples
Contexto	Tudo o que é enviado ao modelo de IA junto com o pedido. É o que ele "sabe" naquele momento. Não persiste entre sessões: o que não for reenviado, o agente não sabe.
Token	A unidade em que o contexto é cobrado — mais ou menos um pedaço de palavra. Em português, uma estimativa usável é 1 token ≈ 3 caracteres . É por isso que o kit orça em caracteres: caractere se conta com um script, token não.
Contexto-fonte	O arquivo <code>a_context_source.md</code> : a única fonte de verdade sobre o estado do projeto, e o único arquivo que <i>toda</i> sessão carrega. Tem teto de 4.000 caracteres, cobrado por script.
Delta	Só o trecho que mudou, em vez do arquivo inteiro reescrito. Pedir delta é a regra que mantém o diff revisável e o custo baixo.
Portão	O critério objetivo que separa "pronto" de "não pronto". Pode ser automático (um script que reprova o commit) ou manual (uma seção de checklist que o dono roda).
Skill / agente	Um arquivo de texto que define um papel para a IA: o que ela faz, o que ela <i>não</i> faz, que arquivos recebe e qual portão precisa cumprir. São 24 no kit, e vale a regra de uma por sessão .

Os cinco identificadores rastreáveis

D-NN decisão fechada · **Q-NN** pendência que é do dono, não do agente · **QA-NN** achado de revisão · **T-NN** tarefa · **A-NN** ação que só o dono pode executar na máquina real. Todos são cobrados pelo script: identificador citado tem de existir, e não pode se repetir.

4. A estrutura: cinco pastas com responsabilidades separadas

A organização não é estética. Cada pasta responde a uma pergunta diferente, e é isso que permite dizer *qual arquivo uma sessão carrega e qual ela nunca carrega* — que é a base da economia de contexto.

Pasta	Pergunta que responde	Conteúdo principal
a_context/	A verdade: o que este projeto é e em que pé está	contexto-fonte (≤ 4.000 caracteres), plano de módulos, registro de decisões, temas de domínio lidos sob demanda
b_process/	Como se trabalha	roteiro das fases, checklist de portões, backlog de tarefas, aprendizados, e a pasta skills/ com os 24 agentes
c_technical_docs/	Como operar	guia do Obsidian, caso de referência
d_history/	O que aconteceu	changelog datado — pode crescer à vontade, porque nenhuma sessão o carrega
e_qa/	Que evidência sustenta	relatórios de revisão com data e hora no nome, decisões arquivadas
scripts/	O que a máquina cobra	o portão automático, os testes, o instalador do hook, o criador de projetos

Além das pastas, três arquivos na raiz carregam papéis distintos, e essa separação é deliberada:

- **README.md** — o único lugar com os "porquês" e com os limites. Ele é para humanos.
- **INDEX.md** — o mapa de navegação. Aponta e sai da frente.
- **CLAUDE.md** — o **contrato de leitura do agente**. A ferramenta de IA carrega esse arquivo sozinha em toda sessão, e ele diz exatamente o que ler, em que ordem, e o que **nunca** ler. Ele existe para que o contexto-fonte não precise gastar o próprio orçamento explicando como ser lido.

Por que isso importa para a banca

A pergunta "como vocês impedem o agente de ler o repositório inteiro?" tem uma resposta concreta: existe um contrato de leitura versionado, com uma tabela de *ler quando*, e uma linha dizendo que o changelog e a pasta de evidências são **nunca**. Sem isso, um agente com acesso à pasta lê tudo e paga por tudo.

5. As 7 regras que valem em todas as fases

Cada regra existe porque uma coisa deu errado antes. O motivo está junto, porque regra sem motivo é a primeira a ser abandonada quando aperta.

#	Regra	O motivo real
1	Contexto com orçamento em número. O contexto-fonte tem teto de 4.000 caracteres, medido por script e atualizado por substituição.	" ≤ 1 página" sem número virou, num projeto real, um parágrafo-parede de ~640 tokens lido em toda sessão. Limite sem número não é limite.

#	Regra	O motivo real
2	Histórico fora do contexto. O que é datado vai para o changelog, que nenhuma sessão carrega.	173 linhas de histórico que nenhuma sessão pagou. Funcionou exatamente como desenhado.
3	Delta, nunca regenerar. Só o trecho alterado — em documento e em código.	Regenerar foi o maior custo dos projetos anteriores: diff ilegível, revisão que ninguém faz, erro que passa.
4	Decisão rastreável. Assunto fechado vira D-NN em duas frases; bug vira QA-NN citado no commit; pendência do dono vira Q-NN.	Sem registro, o que morreu volta. O script cobra: identificador citado tem de existir e não pode repetir.
5	Nada entra sem portão. Critério objetivo definido no dia 1. Rejeição registrada vale tanto quanto adoção.	"Parece bom" não é critério, e sem portão a IA sempre acha que terminou.
6	Estado mora num lugar só. Versão, métrica e contagem vigentes ficam apenas no contexto-fonte; todo outro documento aponta para lá.	Num projeto real o estado vivia em 4 arquivos e divergiu de verdade — dois cards vizinhos do mesmo quadro citando números de versões diferentes.
7	Observe antes de construir. Parser ou integração só com uma amostra real da estrutura na mão.	Chutar a estrutura de uma fonte de dados custou 6 ciclos de revisão num projeto e 6 versões de schema em outro.

6. O fluxo: sete fases, cada uma com seu portão

O roteiro é uma linha do dia 1 à entrega. Cada sessão pressupõe que a anterior passou no portão — é isso que impede o projeto de avançar sobre uma base que ninguém verificou.

Fase	O que acontece	Agente usado	Portão que fecha
0. Contexto 1 sessão	O agente faz até 5 perguntas e devolve o contexto-fonte preenchido. Se o projeto já existe, o mapa se faz do código , não da memória do dono.	bootstrap-contexto (ou adoção de projeto existente)	O script passa (≤ 4.000 caracteres) e o dono lê o contexto inteiro e concorda com cada linha
1. Forma e plano 3 sessões	Decide a arquitetura; gera o plano com módulos, contratos e portões; e — em sessão separada — confere se os documentos contam a mesma história.	arquitetura-monolit o planejador consistência-artefatos	Para cada módulo: "outro agente implementaria isto lendo só o contrato?". Aprovado = plano congelado como decisão
2. Dados e domínio 1 sessão por módulo	Schema, regra de negócio, migrações. Os invariantes são escritos como frases verificáveis antes do código.	backend-domínio (dados-análise antes, se o projeto nasce de fonte externa)	Migração roda em banco vazio; invariantes testados inclusive tentando violá-los direto no banco; transação não deixa efeito parcial
3. Borda, UI e acesso 1 sessão por módulo	Na ordem: acesso, borda, tela.	autenticação backend-bff frontend-uiux	Cada rota sensível testada sem sessão; falha parcial explícita; 4 estados por tela; fluxo crítico no menor viewport
4. Testes e revisão 2 sessões, em ordem	Primeiro os testes; depois, em sessão separada , uma revisão que tenta quebrar o sistema. Repete-se o par até o placar de crítico/alto zerar.	testes guardrails-review	Suíte verde na máquina do dono, rodando duas vezes com o mesmo resultado; relatório de revisão registrado com 12 frentes percorridas e cada achado com reprodução
5. Empacotar e operar 1-2 sessões	Container, deploy, e — se o sistema roda continuamente — uma sessão de observabilidade antes de entregar.	iac-docker-terraform observabilidade	Sobe em ambiente limpo; derrubar e subir preserva os dados; versão consultável em tempo de execução; nenhum segredo na imagem
6. Entrega 1 sessão	Empacota e confere o que sai.	revisão-entrega	Pacote aberto e conferido; nenhum segredo, dependência ou banco dentro; estado numérico só no contexto-fonte

O detalhe que mais impressiona banca: o critério de saída do laço de QA

A fase 4 manda repetir teste e revisão "até zerar". Isso, sozinho, é um laço infinito. Então o kit define o critério de parada: **se o placar de crítico e alto não cair em três passagens consecutivas, o laço de QA para** e abre-se uma sessão de consistência ou de replanejamento. O raciocínio: achado que reaparece três vezes não é bug, é sintoma de plano errado — e continuar revisando queima sessões atacando o efeito. O sinal é objetivo e sai dos relatórios que a própria fase já escreve; não depende de o agente se autoavaliar.

Depois da entrega, o roteiro vira gatilho

A entrega fecha a construção, não o projeto. Daqui em diante não há mais uma linha, e sim uma tabela de situação → agente:

A situação é...	O agente é...	E o portão que não se negocia
"quebrou", "deu erro"	depuração-diagnóstico	reprodução determinística antes de qualquer edição; causa provada ligando e desligando; teste de regressão citando o QA-NN
"está lento"	performance	baseline medido antes; gargalo apontado por ferramenta, não por intuição; uma mudança por vez
"não sei o que houve ontem"	observabilidade	o dono responde às três perguntas sem abrir o código; nada sensível em log
subir biblioteca, CVE	dependências-supply-chain	uma atualização por vez com a suíte verde entre elas; CVE tratada, mitigada com prova, ou aceita com decisão registrada
ideia de melhoria	auditor-evolução	lista-morta percorrida antes; portão escrito antes do experimento

Duas regras de encaminhamento que evitam sessão desperdiçada

"Está lento" ≠ "está lento desde ontem". O segundo é depuração, não performance: algo mudou, e achar o quê é mais barato que otimizar o que já estava rápido.

Precisou depurar duas vezes o mesmo sintoma? O que faltou foi observabilidade. A terceira sessão de arqueologia custa mais que instrumentar de uma vez.

7. O ciclo de uma sessão — o que se repete 90% do tempo

As fases dão a ordem geral, mas o dia a dia é este ciclo, sempre igual:

```
uma skill (o papel) + o CONTEXTO-FONTE + só o arquivo do momento
|
+--> pedir DELTA (só o que muda)
|
+--> rodar o PORTÃO — na máquina do dono, não no sandbox
|
+--> registrar D-NN (decisão) / QA-NN (achado) / Q-NN (pendência do dono)
|
+--> reescrever "Estado atual" do CONTEXTO por SUBSTITUIÇÃO
|
+--> linha datada no CHANGELOG (que nenhuma sessão carrega)
|
+--> commit citando os identificadores: TIPO: o que mudou (D-NN/QA-NN)
```

Repare no que **não** entra na sessão: o repositório inteiro, o changelog, os relatórios de revisão, as outras 23 skills. A sessão recebe três coisas e só três.

O passo mais pulado, e o que se fez a respeito

O fecho de sessão é onde o processo mais vaza — pular um passo é o que faz o estado divergir e o histórico sumir. Por isso ele virou um **modelo de um clique** no Obsidian, com cada item em caixa de seleção: fica visível o que faltou. Atacar o passo mais frágil com a menor fricção possível é uma decisão de projeto, não um detalhe.

8. Os 24 agentes

O que é, tecnicamente, um agente aqui

Cada agente é uma pasta com um arquivo `SKILL.md`. O arquivo tem um cabeçalho com **nome** e **descrição**, e um corpo com quatro seções obrigatórias — obrigatórias no sentido literal: o script reprova o commit se faltar uma.

- **Papel** — quem a IA é nessa sessão.
- **Contexto que você recebe** — exatamente quais arquivos entram. É o oposto de "leia o repositório".
- **Limites** — o que o agente **não** faz mesmo tendo sido escolhido certo. Esta seção é a que impede o agente de consertar coisas fora do escopo da sessão.
- **Saída** — o artefato que sai, para o dono saber o que esperar.

A **descrição** tem uma exigência incomum: precisa dizer quando a skill dispara **e quando não dispara** ("Não use para X — isso é da skill Y"). Sem essa fronteira negativa, duas skills disputam a mesma tarefa e a errada ganha metade das vezes. O script avisa quando falta.

Uma medição que vale citar

Ao comparar este kit com outro conhecido, mediu-se: **21 de 24** descrições daqui já diziam quando *não* escolher a skill (contra 0 de 20 do outro), mas **0 de 24** diziam o que a skill não faz depois de escolhida (contra 15 de 20 lá). Cada kit tinha metade da resposta — e a seção **Limites** nasceu disso, agora cobrada por script.

Os 24, por grupo

Grupo	Agentes	O que o grupo protege
Fases (7)	bootstrap-contexto · adoção-projeto-existente · planejador · consistência-artefatos · auditor-evolução · revisão-entrega · retrospectiva	Que o projeto comece com contexto orçado, plano congelável e documentos que contam a mesma história
Arquitetura (2)	monolito (<i>padrão</i>) · microserviços	Que a forma seja decidida antes de existir código. A de microserviços tem portão de existência : reprova se não houver time e observabilidade — e reprovar é o sistema funcionando
Backend (3)	domínio · bff · integração síncrona	Invariantes, dinheiro em inteiro, transação, timeout, retentativa segura
Frontend (2)	ui/ux · micro-frontend	Quatro estados por tela, mobile-first, erro em linguagem de gente
Transversais (6)	autenticação · infraestrutura · testes · guardrails-review · dependências · privacidade	Nega por padrão, rollback testado, regressão amarrada ao achado, licença e CVE, dado pessoal com finalidade escrita
Sistema vivo (3)	depuração · performance · observabilidade	O projeto passa muito mais tempo sendo mantido que sendo criado — e a versão antiga do kit só tinha agente para a criação
Dados (1)	dados-análise	Amostra real antes do parser; ausente não é zero; número com incerteza; zero vazamento entre treino e teste

Como eles se combinam

```
arquitetura-* (uma vez, na Fase 1)
|
+-- backend-domínio --> backend-bff --> frontend-uiux
    |
    +----- testes -----+
                |
            guardrails-review (antes de entregar)
                |
            iac-docker-terraform (empacotar e subir)
```

A ordem entre eles importa, e é aí que está boa parte do valor: dados-análise entra **antes** de backend-domínio quando o projeto nasce de uma fonte externa — é ela que traz a amostra real sem a qual o schema é chute. Privacidade entra **junto** com o schema, não depois: coluna criada sem finalidade escrita vira obrigação permanente, e retenção retroativa é migração dolorosa.

A regra que sustenta tudo: uma skill por sessão

Duas skills na mesma sessão são duas responsabilidades disputando o mesmo contexto — e é também pagar duas vezes pela mesma instrução. Numa versão anterior existia uma pasta de prompts separada das skills; mediu-se **27% de sobreposição** entre o prompt de revisão e a skill de guardrails, e os dois eram carregados juntos. Os prompts foram absorvidos pelas skills: um mecanismo só.

9. A economia de contexto — o diferencial central

Este é o capítulo que a banca vai querer ouvir com número. A tese é simples de enunciar: **o custo de um projeto assistido por IA não está no que o modelo escreve; está no que ele relê a cada sessão.** Um arquivo de 2.000 caracteres carregado em 40 sessões custa 40 vezes, e ninguém percebe porque cada sessão parece barata.

Os três mecanismos

1. **Orçamento em número, cobrado por máquina.** O contexto-fonte tem teto de 4.000 caracteres — cerca de 1.300 tokens. Não é uma recomendação: o script reprova o commit acima disso, e avisa a partir de 3.600 para que o corte seja feito com calma, e não no meio de uma sessão de trabalho.
2. **Três níveis de leitura, declarados por escrito.** O contrato de leitura divide os arquivos em *sempre* (o contexto-fonte), *sob demanda* (o tema que a tarefa tocar) e **nunca** (changelog e evidências). O terceiro nível é o que mais economiza, e é o que um agente com acesso à pasta faria errado sozinho.
3. **Delta em vez de regeneração.** Vale para documento e para código. Além do custo, é o que mantém o diff pequeno o bastante para alguém revisar de verdade.

Os números medidos

Origem: análise de 22/07/2026 sobre um projeto real construído com a versão anterior do kit. Tamanhos contados nos arquivos; tokens estimados a 3 caracteres por token em português.

O que era carregado	Antes (medido)	Depois (garantido)	O que mudou
Prompt do papel	~620 a 1.150 tokens	~280 a 560 tokens	O texto explicava o motivo histórico da regra ao dono. O motivo foi para o README; a instrução ficou imperativa. ~45% mais barato, mesma função.
Contexto-fonte	~1.580 tokens, sem teto efetivo	≤ ~1.300 tokens, garantido	O "Estado atual" era um parágrafo-parede de 1.930 caracteres com mais de 15 referências, relido em toda sessão. Virou 6 marcadores de formato fixo, cobrados por script.
Registro de decisões	~7.700 tokens (23.101 caracteres)	~2.000 tokens	Era carregado inteiro nas sessões de evolução — o maior custo recorrente do sistema. Agora a linha tem no máximo 2 frases e a evidência longa vira nota linkada.
Changelog	carregado junto com o resto	zero — nenhuma sessão o carrega	173 linhas de histórico que deixaram de ser pagas.

A frase para a banca

"Em dezenas de sessões, a diferença é da ordem de **centenas de milhares de tokens só em contexto fixo** — sem contar o retrabalho evitado." E a ressalva que dá credibilidade: esses valores são **calculados** a partir de tamanhos de arquivo reais, com a conversão de 3 caracteres por token; não são fatura de API.

O detalhe elegante: o teto mede conteúdo, não formatação

Um caso real vale como exemplo do rigor do método. Num projeto, o formatador de Markdown do editor alinhou as colunas de uma tabela ao salvar e somou **2.048 caracteres de espaço em branco puro** — 17% do arquivo, sem uma palavra nova. O portão passou a reprovar um commit que só respondia a uma pergunta. O diagnóstico foi que o teto estava medindo *formatação*. A correção: a medição passou a descontar o alinhamento de tabela. É o tipo de defeito que só aparece quando o limite é cobrado por máquina — e que, sem a máquina, teria virado "o processo é chato" e o abandono da regra.

10. Os guardrails: o que a máquina julga e o que ela não julga

A palavra *guardrail* aqui significa proteção automática — algo que impede o erro de passar sem depender de alguém lembrar. O kit tem duas camadas, e é honesto sobre o tamanho de cada uma.

Camada 1 — o portão automático

Um script em Python puro, sem nenhuma dependência externa, instalado como *hook* de pre-commit: ele roda sozinho a cada commit e, se reprovar, o commit não acontece. Hoje ele julga **14 situações que reprovam** e **16 que avisam**.

Ele reprova quando...	Ele avisa quando...
orçamento do contexto estourado registro de decisões acima do teto o mesmo estado em dois arquivos (fonte única) mais tarefas em andamento que o limite declarado sobra de editor versionada (.bak, .tmp) skill sem nome, sem descrição ou fora do esquema link interno apontando para nota que não existe segredo versionado — na árvore e no histórico gitignore sem cobertura mínima de segredo identificador citado que não existe identificador duplicado "em andamento" divergindo entre backlog e contexto tarefa apontando módulo que não existe no plano	o contexto está perto do teto o registro está perto do teto há módulo no plano sem nenhuma tarefa o portão automático não está instalado há nota que ninguém linka há tema de domínio fora do mapa de leitura a descrição de uma skill não diz quando <i>não</i> usá-la a sessão não declarou qual agente rodou um número declarado diverge do arquivo medido uma pergunta do dono está aberta e ausente do contexto um achado grave está aberto há mais de 14 dias ainda há texto de modelo por preencher

Camada 2 — os portões humanos

O checklist tem 118 itens e as skills trazem mais 166: **284 no total**. O script julga **30 deles** — **cerca de 11%**. O resto depende de o dono rodar a seção certa.

Este é o argumento mais forte do projeto, e é uma limitação

A frase acima está escrita no README e é **cobrada por um teste automatizado**: se alguém acrescentar uma checagem sem atualizar o número, a suíte reprova. O motivo é que essa frase já envelheceu uma vez — dizia 188 itens e 18 julgados quando o real era 277 e 23. Transformar a declaração mais desconfortável do produto num invariante testado é uma decisão de projeto rara, e é a que mais impressiona quem entende do assunto.

Vale saber dizer isto em voz alta: **é um kit de disciplina com algumas travas automáticas, não um sistema que impede erro**. Bancas confiam mais em quem declara o tamanho da própria cobertura do que em quem promete cobertura total.

Camada 3 — os testes do próprio processo

O portão também é testado. São **57 testes**, só com biblioteca padrão do Python, rodando em integração contínua no Linux e no Windows a cada envio. Dois deles merecem destaque:

- **Uma isca por checagem.** Cada uma das 14 falhas tem um caso concreto que ela *precisa* pegar. Se a checagem parar de checar, a isca passa e o teste reprova.

- **O portão do portão.** Um teste compara a lista de iscas com a lista de falhas do script: falha nova sem isca reprova. Isso fecha a classe inteira do defeito, em vez de consertar um caso.

A motivação é concreta: uma das checagens tinha **parado de checar em silêncio**. Ela descartava o texto escrito entre crases antes de procurar os identificadores — e a casa escreve os identificadores entre crases. Das 341 citações de um projeto real, **300 estavam invisíveis**: o portão enxergava 12% e imprimia verde sobre o resto.

11. Rastreabilidade e a "lista-morta"

Toda decisão fechada vira uma linha numerada, com data, status, a decisão em no máximo duas frases e um link para a evidência longa. O registro é **append-only**: nunca se edita uma linha antiga; reverter é escrever uma linha nova que diz que substitui a anterior.

#	Data	Status	Decisão (curta)
D-01	2026-08-06	ADOTADO	Forma = aplicação estática, sem backend
D-06	2026-08-06	REJEITADO	Reaproveitar o backend da versão 1
D-57	2026-08-12	ADOTADO · SUPERSEDE	Cai o portão "sem par repetido"

Por que registrar o que foi REJEITADO

Esta é a ideia central do método e o ponto que melhor diferencia o projeto de um changelog comum. Um changelog registra o que venceu. Aqui registra-se também **o que perdeu, e o número que o matou**. O motivo é específico do trabalho com IA: sem esse registro, o agente re-propõe na sexta a alternativa que foi descartada na segunda, e a discussão inteira se repete — pagando contexto e tempo de novo.

A prova de que o mecanismo funciona sob pressão

Num projeto real, o registro chegou perto do teto e foi preciso arquivar linhas antigas. As rejeitadas seriam as candidatas óbvias — elas estão "mortas". Em **duas passagens diferentes** de arquivamento, elas foram **preservadas de propósito**, com a razão escrita: "são a lista-morta que a sessão de evolução varre". O mecanismo se defendeu da pressão do próprio orçamento. Isso não é documentação decorativa.

Os outros identificadores completam a rastreabilidade: **Q-NN** separa o que é decisão do *dono* (regra de negócio, rumo) do que é decisão do agente — e o agente é instruído a parar e registrar em vez de escolher; **QA-NN** amarra cada achado de revisão à correção e ao teste de regressão que o guarda; **T-NN** e **A-NN** separam tarefa do agente de ação que só o dono pode executar na máquina real.

12. O Obsidian e o valor que ele acrescenta

O que é

O Obsidian é um aplicativo gratuito de notas que trabalha diretamente sobre uma pasta de arquivos Markdown do seu computador — não é um serviço na nuvem e não tem formato próprio. "Vault" é como ele chama essa pasta. Aqui, **o vault é a raiz do repositório git**: as mesmas notas que a pessoa lê no Obsidian são as que o agente de IA lê e as que o git versiona.

O que ele acrescenta ao processo

Recurso	O que resolve na prática
Links internos [[assim]]	O roteiro linka direto a skill de cada fase; o plano linka a decisão que o congelou. Navegar deixa de exigir saber o caminho do arquivo.

Recurso	O que resolve na prática
Backlinks	No rodapé de cada nota aparece quem aponta para ela. É como se descobre todo lugar que cita uma decisão antes de mudá-la — e é o mesmo critério que o script usa para decidir o que pode ser arquivado.
Grafo	Mostra o vault como um mapa, colorido por tema, com o índice, o contexto-fonte e as skills como centros. Serve para enxergar nota órfã e aglomerado inesperado.
Modelos (templates)	O de maior valor. Decisão, achado de QA e fecho de sessão entram com um atalho de teclado. Como o fecho de sessão é o passo mais pulado do processo, reduzir sua fricção a um clique ataca exatamente o ponto por onde o estado começa a divergir.
Busca e etiquetas	Procurar por D-, Q- ou QA- salta direto para qualquer item rastreável.

O ponto que é preciso saber defender

Nada no pipeline depende do Obsidian

É tudo Markdown puro. As skills funcionam em qualquer ferramenta de IA, o portão é Python sem dependências, e o repositório abre normalmente em qualquer editor. O Obsidian é uma **camada de navegação opcional** — se a banca perguntar "e se a equipe não usar Obsidian?", a resposta é: continua funcionando igual, perde-se conforto, não capacidade.

Um detalhe de engenharia que vale citar: a pasta de configuração `.obsidian/` **é versionada de propósito** — é o que faz o vault abrir já configurado para quem clonar o repositório, com favoritos, modelos e cores do grafo prontos. A única exceção é o arquivo de espaço de trabalho, que muda a cada abertura e está no gitignore. E a pasta fica fora da validação do script, para não gerar aviso falso — porque **aviso falso ensina a ignorar aviso**, que é uma regra do próprio kit.

13. O papel do dono — o que a IA não faz por você

O método é explícito sobre a fronteira. Isso importa numa apresentação porque a pergunta "então a IA faz tudo?" vem sempre.

Papel	O que significa
Guardião do portão	Aceitar uma entrega é rodar a seção certa do checklist. Se falhar, devolve-se pedindo <i>delta</i> — nunca "refaz tudo".
Decisor	Toda pendência Q-NN é do dono. O agente não muda regra de negócio nem rumo: ele registra a pergunta e para.
Operador da máquina real	Testes oficiais, migrações, deploy e envio ao repositório remoto. O sandbox do agente é indicativo, nunca portão .
Fonte dos dados manuais	O que o dono não preencher fica como lacuna declarada — jamais inventada. Essa é uma regra dura do kit: o agente não fabrica dado, fonte ou número.

As frases de segurança

O kit lista perguntas curtas que o dono deve fazer ao agente. Elas economizam sessão inteira e funcionam como um roteiro de defesa:

- "Isso passou no portão? **Mostra o número.**"
- "Cadê o D-NN?"
- "Me manda **só o delta.**"
- "Rodou na **minha máquina** ou no sandbox?"
- "Viu uma **amostra real** antes de escrever esse parser?"
- "Isso muda alguma **decisão ou número?**"

14. A evidência de campo

Um processo que só fala bem de si mesmo não convence. Por isso o kit foi **medido** contra um projeto real construído com ele — um jogo de disputa de pênaltis, aplicação estática sem servidor, 46 commits em 4 dias de trabalho e 34 sessões registradas. Dois cuidados dão credibilidade à medição:

- Os agentes que construíram o projeto **não sabiam** que ele era um teste do kit.
- O critério de sucesso foi escrito, datado e assinado por hash **antes** de qualquer arquivo do projeto ser aberto. Avaliação que escolhe o critério depois de ver o resultado não vale nada.

O que se mediu	Resultado
Commits citando um identificador rastreável	45 de 46 = 98%
Taxa de regeneração de arquivo (o oposto de delta)	~0% — 3 candidatos, os 3 conferidos à mão, nenhum era reescrita
Decisões rejeitadas registradas com motivo	6 , 15% das linhas vivas; nenhuma reapareceu

O que se mediu	Resultado
Perguntas do dono abertas e respondidas	11 abertas, 8 respondidas (73%)
Achados de revisão	16, em 3 passagens; 4 dos 5 críticos fechados
Sabotagens do portão (ele reprova de verdade?)	3 de 3 reprovadas , incluindo bloquear um commit com chave de acesso
Módulos do plano sem tarefa no backlog	0

E o custo, que também foi medido

A parte que dá credibilidade a um relatório é a que não favorece o autor:

- **15% das sessões** (5 de 34) foram gastas administrando o orçamento do próprio processo, sem uma linha de produto.
- **43% dos commits** não tocam código.
- **9 defeitos conhecidos** ficaram abertos por causa da regra de escopo — um deles de uma única linha de CSS.
- Um modo do produto ficou parado dias esperando uma decisão do dono.

O achado mais valioso foi contra o próprio kit

A medição encontrou uma checagem do portão que **tinha parado de checar em silêncio** e enxergava 12% dos casos que anunciava cobrir. Descobrir isso vale mais que confirmar que o processo funciona — e a correção não foi consertar aquele caso, foi criar o teste que impede a classe inteira do defeito de voltar.

15. Onde o kit não serve (limites declarados)

Estar escrito no próprio README é parte do argumento: nenhuma ferramenta serve para tudo, e declarar a fronteira é o que faz alguém confiar no que está dentro dela.

Contexto	Serve?	O que trava
Aplicação pequena ou média, com um dono	Sim — é o alvo	Nada
Projeto que já existe	Sim	Começa por uma skill de adoção que produz o mapa a partir do código. Quanto maior o sistema, mais o teto de 4.000 aperta
Time de 2 a 4 pessoas	Parcialmente	O limite de tarefas simultâneas é configurável, mas não há atribuição por pessoa nem junção de decisões concorrentes
Aplicação grande (30+ módulos)	Não	4.000 caracteres não representam 30 módulos
Projeto longo (100+ decisões)	Com atrito	O teto do registro dá cerca de 66 linhas, e o arquivamento exige decisão do dono
Vários repositórios	Não	O kit assume um repositório e um contexto-fonte
Integração contínua e revisão por pares	Não cobre	O único automatismo é o pre-commit

Há ainda um limite técnico honesto: a varredura de segredos é uma rede de arrasto, não uma garantia. Ela cobre 11 famílias de padrão e foi medida contra 8 formatos reais de vazamento (acertou os 8, com zero falsos positivos em 12 iscas) — mas um segredo num formato que ela não conhece passa. A versão anterior dessa varredura detectava **0 de 8**, e foi uma auditoria que revelou isso.

16. Perguntas prováveis da banca, com resposta pronta

“Isso não é só documentação bonita?”

Não, porque uma parte é **cobrada por máquina**. Há um script que reprova o commit em 14 situações, um hook que o roda sozinho e 57 testes que garantem que o próprio script continua funcionando. E o kit declara o tamanho da parte automática: 30 de 284 itens, cerca de 11%. Documentação bonita não reprova commit.

“Qual é o ganho concreto?”

Três, e os três têm número: economia de contexto (o registro de decisões saiu de ~7.700 para ~2.000 tokens por sessão de evolução; o contexto-fonte tem teto garantido); rastreabilidade (98% dos commits do projeto medido citam um identificador); e revisão possível (taxa de regeneração de arquivo perto de zero, o que mantém o diff pequeno).

“Depende de qual IA?”

Não. Os agentes são arquivos Markdown que funcionam instalados na ferramenta ou colados numa conversa qualquer. O portão é Python de biblioteca padrão. O repositório é Markdown puro. O que muda de ferramenta para ferramenta é o conforto, não a capacidade.

“Como vocês impedem a IA de inventar coisa?”

Por três mecanismos combinados: **lacuna declarada fica declarada** — o agente é instruído a nunca inventar dado, fonte ou número; regra de negócio ambígua não é dele para decidir, vira uma pergunta registrada e a sessão para; e o portão final roda **na máquina do dono**, não no ambiente do agente, então “passou” é um fato verificável e não uma afirmação.

“E se o agente errar mesmo assim?”

Erra, e o processo assume isso. A fase de revisão é **adversarial e em sessão separada** — o mesmo contexto que construiu não enxerga o próprio ponto cego. Cada achado vira um identificador com reprodução e correção amarrada a um teste de regressão. E existe um critério de saída do laço, para não revisar para sempre.

“Vocês mediram, ou é opinião?”

Mediu-se. Há uma avaliação de campo com o critério de sucesso escrito e hashado antes de olhar os dados, e ela publica **o custo junto com o ganho**: 15% das sessões do projeto foram manutenção do próprio processo. É uma medição de um projeto só — ataca a falta de evidência, não a resolve.

“Qual a maior fraqueza?”

A escala do orçamento. O teto do registro de decisões dá cerca de 66 linhas, e quando ele enche o arquivamento custa trabalho de verdade. Isso está declarado no README, foi medido no projeto real (a folga projetada era de poucos dias) e a solução — subir o teto — é uma decisão consciente, não um conserto pendente escondido.

“Por que Obsidian?”

Por conforto de navegação e pelos modelos de um clique, que atacam o passo mais pulado do processo. Nada depende dele: é Markdown puro, e o pipeline funciona igual em qualquer editor.

Anexo A — Glossário rápido

Termo	Definição em uma linha
Agente / skill	Arquivo que define um papel para a IA, com limites e portão próprios
Append-only	Só se acrescenta linha; nunca se edita nem apaga o que já foi registrado
Backlog	A fonte única de tarefas, com um limite declarado de tarefas simultâneas
Baseline	A medição de referência tirada antes de mexer em qualquer coisa
Contexto	Tudo o que é enviado ao modelo junto com o pedido
Contexto-fonte	O arquivo de verdade do projeto; único que toda sessão carrega
Delta	Só o trecho que mudou
Frontmatter	O cabeçalho de metadados no topo de uma nota Markdown
Hook de pre-commit	Programa que o git roda antes de cada commit e que pode bloqueá-lo
Invariante	Uma verdade que o sistema não pode violar nunca (ex.: saldo nunca negativo)
Lacuna declarada	Informação que falta e é registrada como faltando, jamais inventada
Lista-morta	O conjunto de decisões rejeitadas, mantido para impedir que voltem
Portão	Critério objetivo que separa pronto de não pronto
Revisão adversarial	Revisão que tenta quebrar, em sessão separada de quem construiu
Sandbox	O ambiente do agente — indicativo, nunca o portão final
Token	Unidade de cobrança do contexto; ~3 caracteres em português
Vault	A pasta de notas do Obsidian; aqui, a raiz do repositório
WIP	Trabalho em andamento; o kit cobra o limite que o próprio projeto declarou

Anexo B — Ficha de avaliação preenchida

Cada nota abaixo vem com a justificativa e o número que a sustenta, para você defender a pontuação item por item. Escala de 0 a 4. Campos que dependem de informação que não está no repositório ficaram em branco de propósito.

Campo	Resposta	Justificativa e evidência
Role	(em branco)	Depende do que a banca entende por "Role". Confirmar antes de preencher.
IDE	Claude Code / Claude Desktop	Ferramenta usada nas sessões do projeto.
LLM	Claude Opus 5 (esforço MAX)	Modelo usado nas sessões do projeto.

Domain

Item	Qual?	Nota	Justificativa
MCP	nenhum	0	Verificado: não há servidor MCP configurado nem menção a MCP em nenhum arquivo do kit ou do projeto. A integração é por skills e por hook de git, não por MCP.
Agent	24 agentes de papel único	4	24 agentes com papel, limites e portão próprios, agrupados por fase, arquitetura, backend, frontend, transversais, sistema vivo e dados. Regra de uma skill por sessão , e a ordem entre eles é documentada.
Skill	24 SKILL.md instaláveis	4	Formato nativo do Claude Code/Cowork, com nome e descrição no cabeçalho. O esquema é cobrado por script : skill sem "Contexto que você recebe", "Limites" ou "Saída" reprova o commit; descrição sem fronteira negativa gera aviso.
Brain	contexto-fonte + registro + contrato de leitura	4	Memória deliberada e orçada: contexto-fonte de 4.000 caracteres como fonte única, registro append-only de decisões com teto de 12.000, temas de domínio lidos sob demanda, aprendizados acumulados e um contrato de leitura que declara o que nunca ler. Limite conhecido e declarado: o teto do registro não escala além de ~66 linhas.

SPC — especificação

Item	Nota	Justificativa
Functional	4	Contexto-fonte com objetivo, restrições inegociáveis e critério de aceite objetivo no dia 1; plano com módulos e milestones; backlog em que todo card tem portão escrito . Uma sessão dedicada (consistência-artefatos) confere se especificação, plano e tarefas contam a mesma história, e reprova critério de aceite sem número.
Technical	4	Stack e restrições da stack declaradas antes do primeiro código; representações obrigatórias (dinheiro em inteiro, data UTC, identificador opaco); contrato por módulo com porta única, dono de estado e portão objetivo; plano congelado por decisão registrada, e mudança de porta exige nova decisão.

Guardrail

Item	Nota	Justificativa
Doc	4	Contrato de leitura carregado em toda sessão; 7 regras com o motivo de cada uma; checklist de 118 itens; seção Limites obrigatória nas 24 skills; templates de decisão, achado e fecho de sessão. E a declaração de cobertura (30 de 284) é cobrada por teste .
Code	4	Portão em Python sem dependências, com 14 falhas e 16 avisos, instalado como hook de pre-commit; varredura de segredo na árvore e no histórico ; integração contínua em Linux e Windows a cada envio. Verificado por sabotagem: 3 de 3 violações plantadas foram reprovadas, incluindo o bloqueio de um commit com chave de acesso.

Test

Item	Nota	Justificativa
Unit	4	57 testes no kit, só com biblioteca padrão, incluindo uma isca canônica por checagem e um teste que exige que toda checagem nova tenha a sua. No projeto medido: 385 testes, com invariantes verificados por sabotagem (reverter a regra à mão reprova o teste).
System	3	O kit testa ponta a ponta os próprios fluxos (criar projeto, atualizar, instalar hook, rodar o portão sobre um repositório montado do zero). O que impede a nota 4 é o lado do projeto: há teste de integração entre módulos, mas nenhuma tela tem teste automatizado — o ambiente roda sem navegador, e a conferência visual ficou como ação manual do dono. A lacuna está declarada no próprio projeto, não escondida.

Other

Item	Nota	Justificativa
GitHub	4	Repositório público sob licença MIT, com integração contínua rodando o portão em Linux e Windows a cada envio, hook de pre-commit instalável por comando, e convenção de commit citando os identificadores — medido em 98% no projeto real.
Planilha	<i>vazio</i>	O projeto não usa planilha, e o item veio sem escala. Confirmar o que a banca espera aqui.
Agnostic	4	Markdown puro e Python de biblioteca padrão, sem nenhuma dependência externa. As skills funcionam instaladas ou coladas em qualquer ferramenta de IA; o Obsidian é opcional; roda em Windows, Linux e Mac, com o encoding testado nos dois sistemas. A única parte específica de ferramenta é o modo "instalado" das skills.
Explanation	—	Um processo reutilizável para construir software com agentes de IA mantendo rigor e gastando pouco contexto. Ele impõe orçamento numérico ao que a IA relê a cada sessão, exige delta em vez de reescrita, registra decisões — inclusive as rejeitadas — e cobra por máquina o que dá para cobrar por máquina, declarando com honestidade o tamanho da parte que continua humana: 30 de 284 itens.